Nagar YuwakShikshanSanstha's
Yeshwantrao Chavan College of Engineering
(An Autonomous Institution affiliated to RashtrasantTukadojiMaharaj Nagpur University)
Hingna Road, Wanadongri, Nagpur - 441 110
NAAC A++
Ph.: 07104-237919, 234623, 329249, 329250 Fax: 07104-232376, Website: www.ycce.edu

Department of Computer Science & Engineering (IoT)

Vision of the Department

To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skillsthrough emerging technologies.

Session 2026-2027

Vision: Dream of where you want.	**Mission:** Means to achieve Vision

Program Educational Objectives of the program (PEO): (broad statements that describe the professional and career accomplishments)

PEO1	**Preparation**	**P: Preparation**	**Pep-CL abbreviation pronounce as Pep-si-lL easy to recall**
PEO2	**Core Competence**	**E: Environment (Learning Environment)**	
PEO3	**Breadth**	**P: Professionalism**	
PEO4	**Professionalism**	**C: Core Competence**	
PEO5	**Learning Environment**	**L: Breadth (Learning in diverse areas)**	

Program Outcomes (PO): (statements that describe what a student should be able to do and know by the end of a program)

Keywords of POs:
Engineering knowledge,Problem analysis, Design/development of solutions,Conduct Investigations of Complex Problems, Engineering Tool Usage, The Engineer and The World,Ethics, Individual and Collaborative Team work,Communication, Project Management and Finance, Life-Long Learning

PSO Keywords: Cutting edge technologies, Research

"I am an engineer, and I know how to apply engineering knowledge to investigate, analyse and design solutions to complex problems using tools for entire world following all ethics in a collaborative way with proper management skills throughout my life." *to contribute to the development of cutting-edge technologies and Research*.

Integrity: I will adhere to the Laboratory Code of Conduct and ethics in its entirety.

Sheikh Wasimuddin
Name of Student

Nagar YuwakShikshanSanstha's
Yeshwantrao Chavan College of Engineering
(An Autonomous Institution affiliated to RashtrasantTukadojiMaharaj Nagpur University)
Hingna Road, Wanadongri, Nagpur - 441 110
NAAC A++
Ph.: 07104-237919, 234623, 329249, 329250 Fax: 07104-232376, Website: www.ycce.edu
Department of Computer Science & Engineering (IoT)

Vision of the Department

To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skillsthrough emerging technologies.

Session	**2026-27 (ODD)**	**Course Name**	**PE-V : Lab : Blockchain Technology**
Semester	**7**	**Course Code**	**23IOT1748**
Roll No	**61**	**Name of Student**	**Sheikh Wasimuddin**

Practical Number	**05**
Course Outcome	**1. Demonstrate basics of distributed systems and cryptographic primitives used in blockchain.** **2. Outline various research aspects for development of blockchain.** **3. Make use of different security protocols to build secure blockchain applications.** **4. Analyze smart contracts and Hyperledger Fabric used in blockchain. 5. Determine the performance of applications developed with Bitcoin and Ethereum blockchain.**
Aim	Implement Solidity Programs Using Variables, Data Types, Operators, Functions, Conditional Statements, and Loops.
Problem Statements	**1. Student Result Smart Contract** Create a smart contract that: • Stores student information • Stores marks in three subjects • Calculates total marks • Calculates the average • Determines pass / fail **2. Electricity Bill Calculator** Create a smart contract that calculates an electricity bill based on units consumed. • Store: Customer Name, Customer ID, Units Consumed • 0 – 100 units → ₹5 per unit • 101 – 200 units → ₹7 per unit • Above 200 units → ₹10 per unit • Write a function to calculate the total bill using conditional statements.
Theory (100 words)	Solidity is a programming language designed for developing smart contracts on blockchain platforms such as Ethereum. It is a high-level, statically typed language that allows developers to write programs for managing digital assets, storing information, and implementing decentralized applications.

Solidity programs are compiled into bytecode and executed by the Ethereum Virtual Machine (EVM).

1. Variables and Data Types

Variables are used to store and manage information in a smart contract. Solidity provides different data types according to the nature of the data.

- uint: Stores unsigned integer values.
- int: Stores signed integer values.
- bool: Stores either true or false.
- string: Stores text.
- address: Stores an Ethereum account or contract address.

Variables may be declared as state variables, which retain their values on the blockchain, or local variables, which are used temporarily during function execution.

2. Operators

Operators are used to perform calculations, comparisons, and logical operations in Solidity.

- Arithmetic operators: +, -, *, /, %
- Comparison operators: ==, !=, >, <, >=, <=
- Logical operators: &&, ||, !
- Assignment operators: =, +=, -=, *=

Operators help smart contracts perform mathematical calculations, compare values, and make decisions based on conditions.

3. Functions

Functions are blocks of code that perform specific tasks in a smart contract. They can accept input parameters and return results. Functions are used to read or modify contract data and implement the main operations of a decentralized application.

Solidity provides different visibility levels:

- public: Can be accessed internally and externally.
- private: Can be accessed only within the same contract.
- internal: Can be accessed within the contract and derived contracts.

<table>
<tr><td></td><td>
external: Can be called from outside the contract.
Functions can also be declared as view when they only read blockchain state, or pure when they do not read or modify blockchain state.
4. Conditional Statements
Conditional statements allow a smart contract to execute different instructions depending on whether a condition is true or false. Solidity mainly supports if, else if, and else statements.
For example, a contract can check whether a student's marks are greater than or equal to 40 and return Pass; otherwise, it returns Fail.
Conditional statements are useful for implementing rules, validating inputs, and controlling the execution of contract functions.
5. Loops
Loops are used to execute a block of code repeatedly. Solidity supports the following loops:
for loop: Used when the number of iterations is known.
while loop: Executes as long as a specified condition remains true.
do-while loop: Executes the block at least once before checking the condition.
Loops can be used for tasks such as calculating sums, processing arrays, and finding factorials. However, excessive use of loops should be avoided because blockchain computation consumes gas, which can increase transaction costs.
</td></tr>
<tr><td>Procedure and Execution

(100 Words)</td><td>
A. Student Result Smart Contract
Implementation Steps:
Open Remix IDE and create a new Solidity file named StudentResult.sol.
Specify the Solidity compiler version using the pragma statement.
Create a smart contract named StudentResult.
Declare variables to store the student name, student ID, and marks in three subjects.
Create a function to enter and store the student information and marks.
</td></tr>
</table>

6. Create a function to calculate the total marks by adding the marks of all three subjects.
7. Create a function to calculate the average marks using the formula:

Average = Total Marks / 3

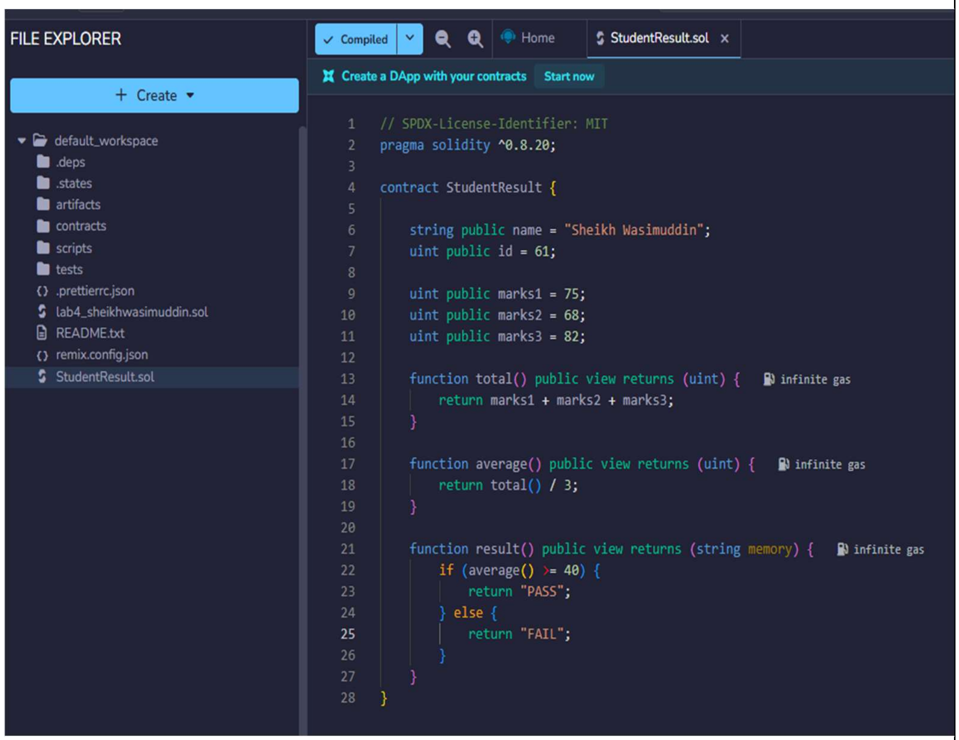
8. Use conditional statements to determine the result:
   - Average ≥ 40 → Pass
   - Average < 40 → Fail
9. Create a function to display the student information, total marks, average, and result.
10. Compile the smart contract using the Solidity Compiler in Remix IDE.
11. Deploy the smart contract using the Deploy & Run Transactions option.
12. Enter the student's details and marks.
13. Call the calculation functions and verify the total marks, average marks, and pass/fail result.

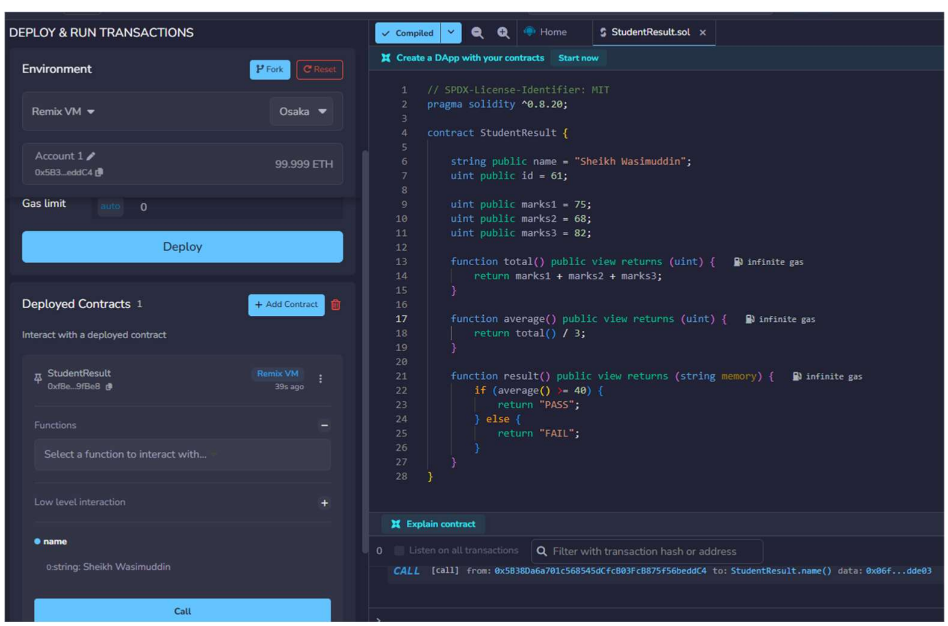
B. Electricity Bill Calculator

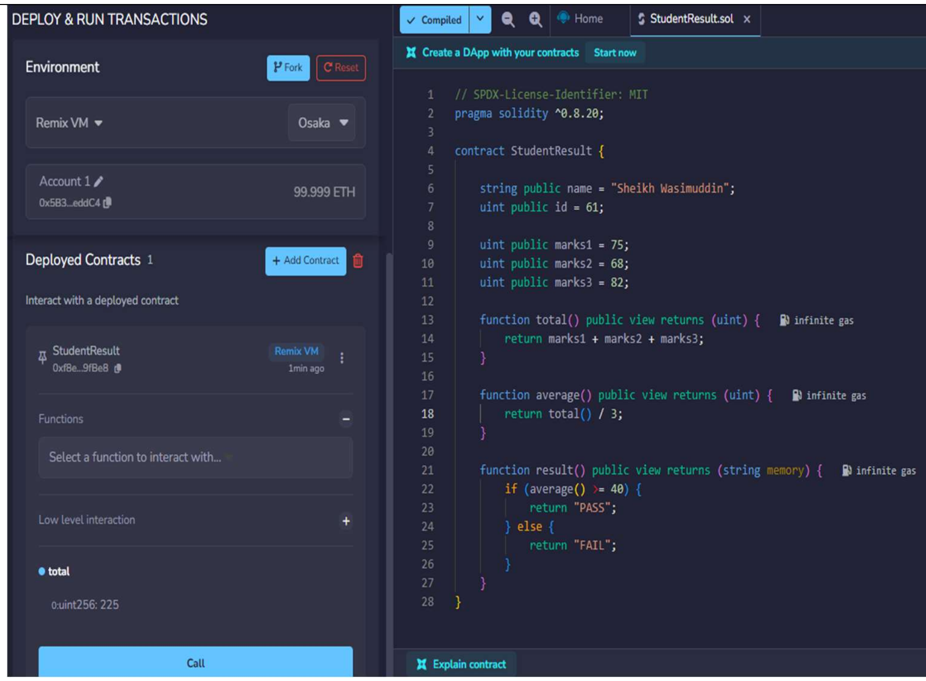
Implementation Steps:

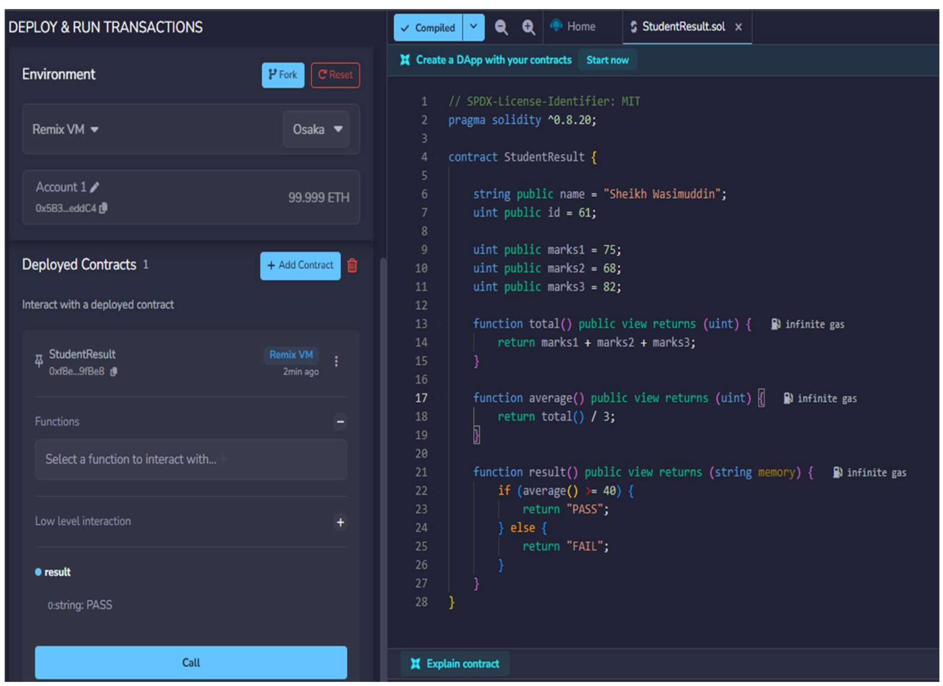
1. Open Remix IDE and create a new Solidity file named ElectricityBill.sol.
2. Specify the Solidity compiler version using the pragma statement.
3. Create a smart contract named ElectricityBill.
4. Declare variables to store the customer name, customer ID, and units consumed.
5. Create a function to enter and store the customer details and units consumed.
6. Create a function named calculateBill() to calculate the electricity bill.
7. Use conditional statements to determine the electricity rate:
   - 0–100 units → ₹5 per unit
   - 101–200 units → ₹7 per unit
   - Above 200 units → ₹10 per unit
8. Calculate the total bill using the selected rate and units consumed.
9. Create a function to display the customer details and total bill.
10. Compile the smart contract using the Solidity Compiler in Remix IDE.
11. Deploy the smart contract using the Deploy & Run Transactions option.
12. Enter the customer details and units consumed.
13. Call the calculateBill() function and verify the total electricity bill.

Student Result Smart Contract Stepwise Screenshots:

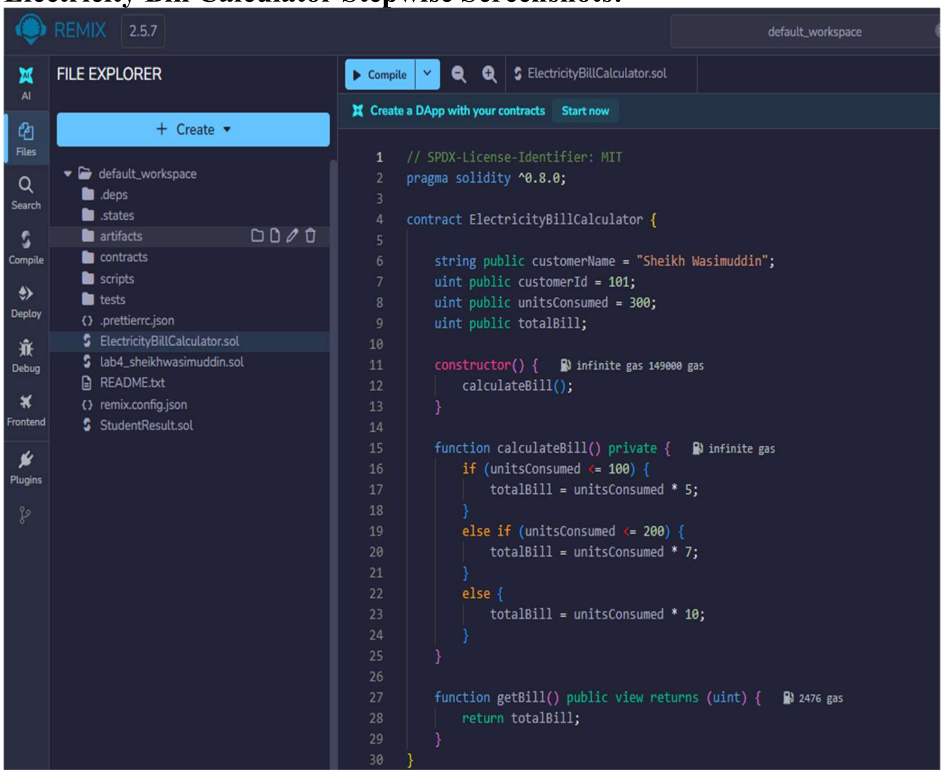


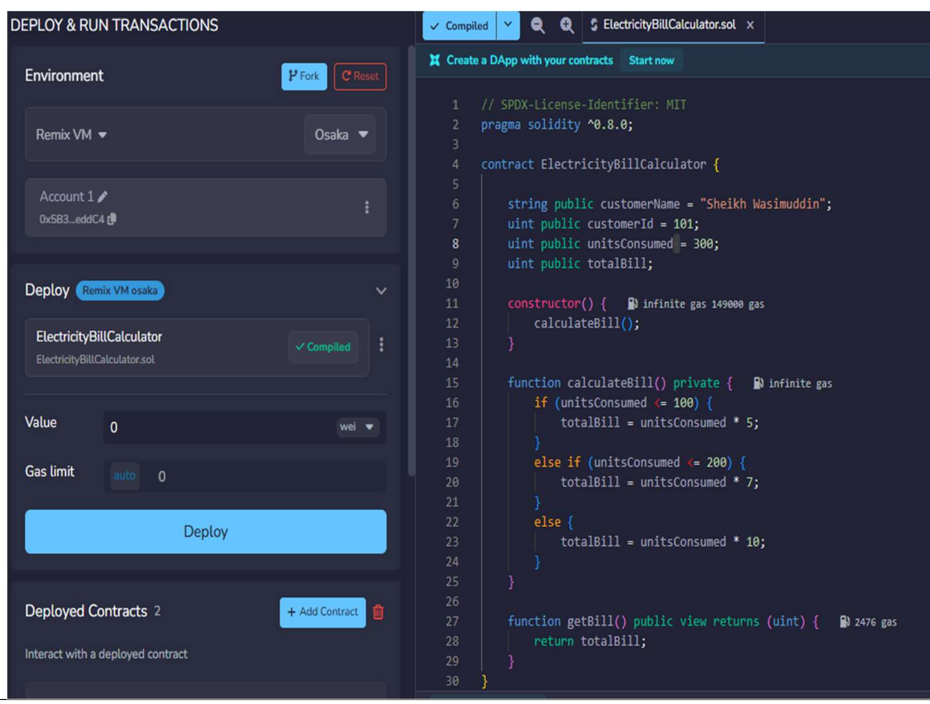


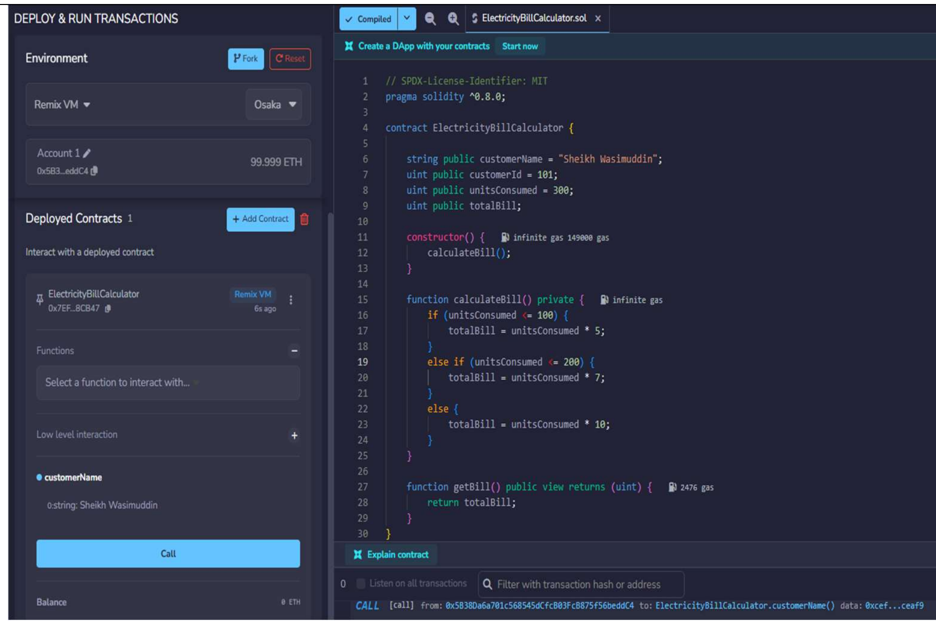

DEPLOY & RUN TRANSACTIONS
Environment
Fork
Reset
Remix VM
Osaka
Account 1
0x5B3...eddC4
99.999 ETH
Deployed Contracts 1
Add Contract
Interact with a deployed contract
StudentResult
0xf8e...9fBe8
Remix VM
1min ago
Functions
Select a function to interact with...
Low level interaction
total
0:uint256: 225
Call
Compiled
Home
StudentResult.sol
Create a DApp with your contracts
Start now
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
contract StudentResult {
string public name = "Sheikh Wasimuddin";
uint public id = 61;
uint public marks1 = 75;
uint public marks2 = 68;
uint public marks3 = 82;
function total() public view returns (uint) {
infinite gas
return marks1 + marks2 + marks3;
}
function average() public view returns (uint) {
infinite gas
return total() / 3;
}
function result() public view returns (string memory) {
infinite gas
if (average() >= 40) {
return "PASS";
} else {
return "FAIL";
}
}
}
Explain contract

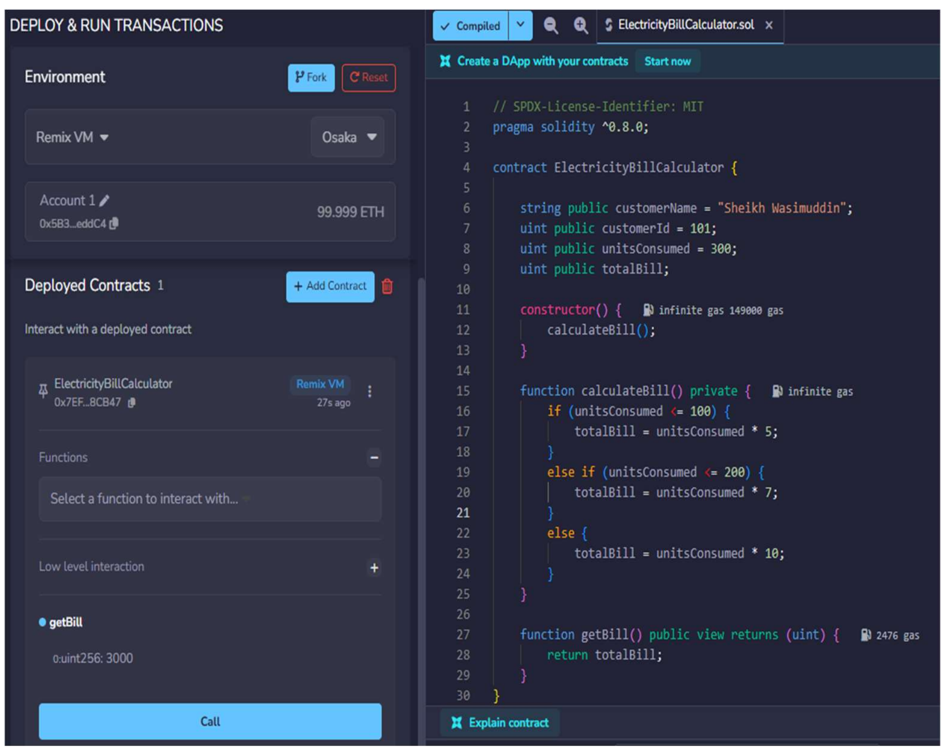

DEPLOY & RUN TRANSACTIONS
Environment
Fork
Reset
Remix VM
Osaka
Account 1
0x5B3...eddC4
99.999 ETH
Deployed Contracts 1
Add Contract
Interact with a deployed contract
StudentResult
0xf8e...9fBe8
Remix VM
2min ago
Functions
Select a function to interact with...
Low level interaction
result
0:string: PASS
Call
Compiled
Home
StudentResult.sol
Create a DApp with your contracts
Start now
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
contract StudentResult {
string public name = "Sheikh Wasimuddin";
uint public id = 61;
uint public marks1 = 75;
uint public marks2 = 68;
uint public marks3 = 82;
function total() public view returns (uint) {
infinite gas
return marks1 + marks2 + marks3;
}
function average() public view returns (uint) {
infinite gas
return total() / 3;
}
function result() public view returns (string memory) {
infinite gas
if (average() >= 40) {
return "PASS";
} else {
return "FAIL";
}
}
}
Explain contract

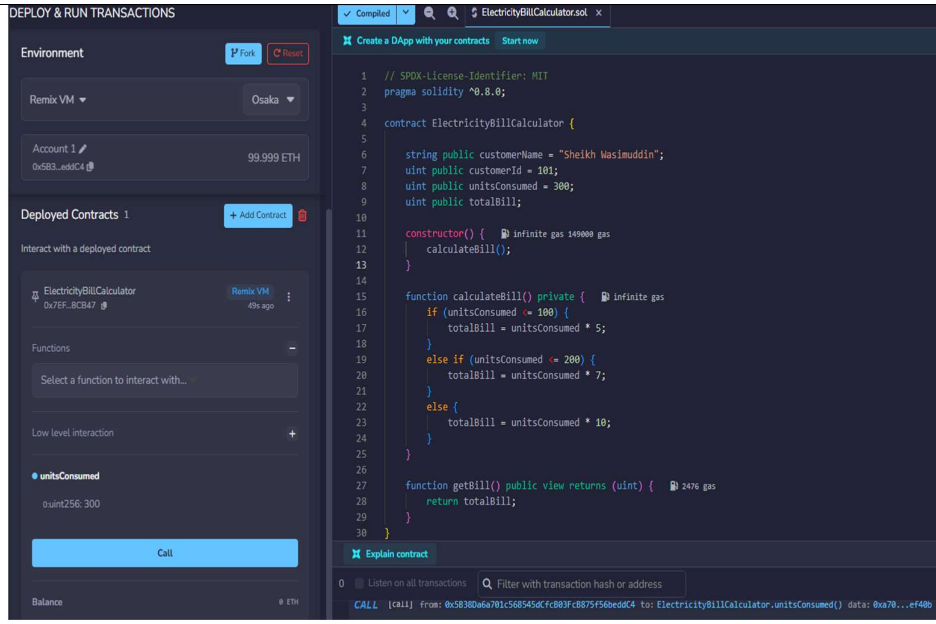
Electricity Bill Calculator Stepwise Screenshots:

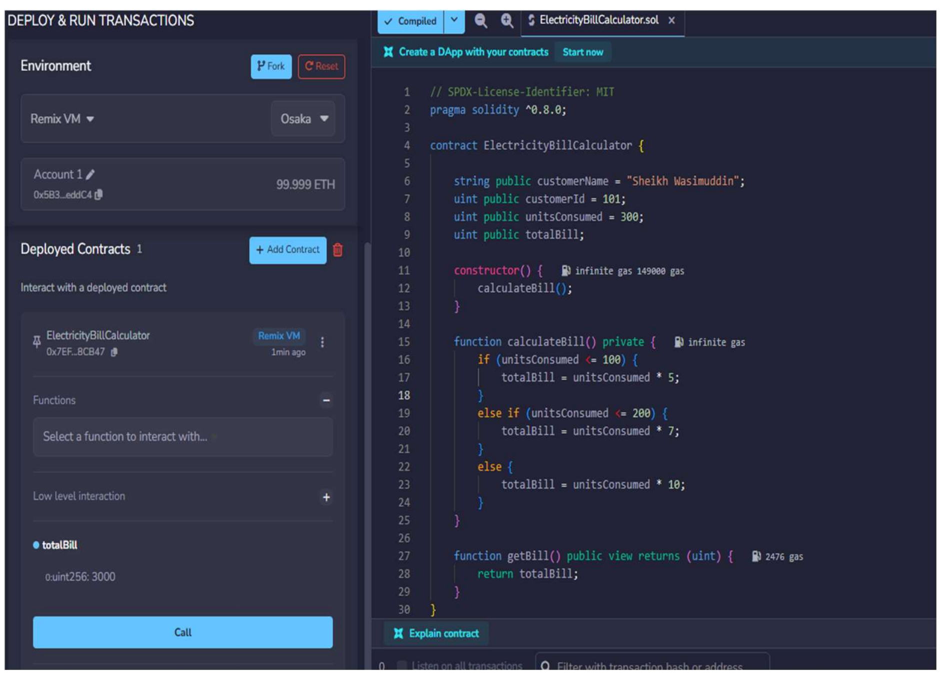





DEPLOY & RUN TRANSACTIONS
Environment
Fork
Reset
Remix VM
Osaka
Account 1
0x5B3...eddC4
99.999 ETH
Deployed Contracts 1
Add Contract
Interact with a deployed contract
ElectricityBillCalculator
0x7EF...8CB47
Remix VM
Functions
Select a function to interact with...
Low level interaction
customerName
0:string: Sheikh Wasimuddin
Call
Balance
Compiled
ElectricityBillCalculator.sol
Create a DApp with your contracts
Start now
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
contract ElectricityBillCalculator {
string public customerName = "Sheikh Wasimuddin";
uint public customerId = 101;
uint public unitsConsumed = 300;
uint public totalBill;
constructor() {
calculateBill();
}
function calculateBill() private {
if (unitsConsumed <= 100) {
totalBill = unitsConsumed * 5;
}
else if (unitsConsumed <= 200) {
totalBill = unitsConsumed * 7;
}
else {
totalBill = unitsConsumed * 10;
}
}
function getBill() public view returns (uint) {
return totalBill;
}
}
Explain contract
Listen on all transactions
Filter with transaction hash or address


DEPLOY & RUN TRANSACTIONS
Environment
Fork
Reset
Remix VM
Osaka
Account 1
0x5B3...eddC4
99.999 ETH
Deployed Contracts 1
Add Contract
Interact with a deployed contract
ElectricityBillCalculator
0x7EF...8CB47
Remix VM
27s ago
Functions
Select a function to interact with...
Low level interaction
getBill
0:uint256: 3000
Call
Compiled
ElectricityBillCalculator.sol
Create a DApp with your contracts
Start now
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
contract ElectricityBillCalculator {
string public customerName = "Sheikh Wasimuddin";
uint public customerId = 101;
uint public unitsConsumed = 300;
uint public totalBill;
constructor() {
infinite gas 149000 gas
calculateBill();
}
function calculateBill() private {
infinite gas
if (unitsConsumed <= 100) {
totalBill = unitsConsumed * 5;
}
else if (unitsConsumed <= 200) {
totalBill = unitsConsumed * 7;
}
else {
totalBill = unitsConsumed * 10;
}
}
function getBill() public view returns (uint) {
2476 gas
return totalBill;
}
}
Explain contract





Conclusion	The practical successfully demonstrated the implementation of two Solidity smart contracts, StudentResult and ElectricityBill, using Remix IDE. The StudentResult contract showed how student details and marks can be stored, total and average marks can be calculated, and the result can be determined

	using conditional statements. The ElectricityBill contract demonstrated how customer details and units consumed can be stored and how the electricity bill can be calculated using different unit rates. Both contracts were compiled and deployed successfully, and their functions were executed to verify the results. The practical helped in understanding Solidity variables, functions, operators, conditional statements, and the basic process of developing and deploying smart contracts on the Ethereum blockchain.
Date	25/08/2026